User Acceptance Testing (UAT) Template

Date	16 August 2026
Project Name	Employee Raise Issue – Record Producer & Service Portal Integration
Maximum Marks	4 Marks

Here is the polished and professionally rewritten version of your **Project Overview, Testing Scope, and Environment Strategy**.

The language has been upgraded to mirror enterprise-level Quality Assurance (QA) and software release documentation standards, aligning user stories with standard Agile practices and utilizing formal technical verification terminology.

---

1. Project Metadata & Executive Summary

Project Attribute	Enterprise Configuration Management Specification
Project Name	Employee Raise Issue – Record Producer & Service Portal Integration
Project Description	Delivery of a centralized, secure internal intake engine within the ServiceNow platform. The application provides end-users with an intuitive Service Portal interface to report workplace anomalies via an encapsulated Record Producer. The architecture enforces structural data integrity via client-side validations, conditional UI Policies, and a dynamic Category–Subcategory dependency matrix, automatically committing

	verified payloads into a dedicated custom table (`u_employee_issue`).
Release Version	Version 1.0 (Base Baseline Release)
Verification Window	15 August 2026 – 15 August 2026

2. Testing Scope & Functional Verification Boundaries

- Features & Capabilities Under Test
- Portal Accessibility: Cross-browser verification of portal navigation and user entitlement to access the "Raise Employee Issue" catalog component.
- Intake Form Execution: Structural evaluation of the Record Producer's field layouts, variable sets, and runtime data handling.
- Contextual Identity Capture: Validation of the systemic capture of reference fields (e.g., Opening User/Requester parameters).
- Data Classification Integrity: Verification of the multi-tier selection behavior across categories and subcategories.
- Dependency Engine Logic: Functional validation of the dynamic cascade filtering between Category and Subcategory selections.
- State Machine Management: Inspection of record state variables and baseline initialization parameters within the transactional intake context.
- Input Constraint Validations: Functional boundary testing of mandatory conditions imposed on the Short Description input field.
- Transaction Processing (Submit): End-to-end verification of submission payload compilation, error avoidance, and acknowledgment rendering.
- Target Persistence Verification: Verification of relational table integrity, confirming variables accurately map and instantiate clean rows within the `u_employee_issue` table.
- UI Engine Compliance: Rigorous structural verification of configured Client Scripts, UI Policies, and visibility/read-only access states.

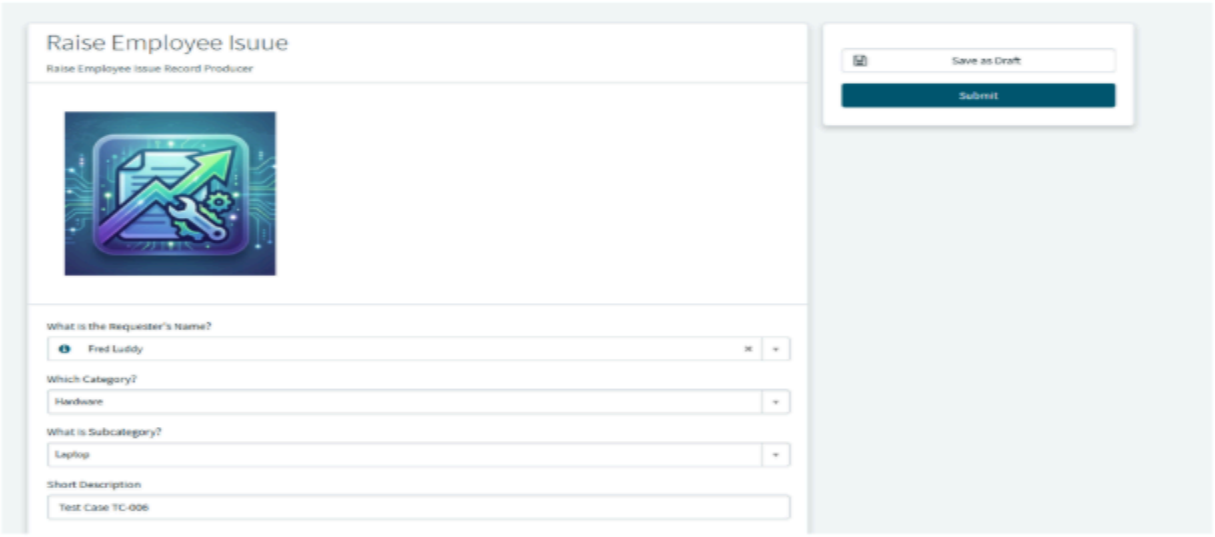
Target User Stories & Acceptance Criteria Evaluated

- US-01 (Access Control): *As an employee, I want to securely access the intake form from the Service Portal so that I can report workplace anomalies via a formalized medium.*

- US-02 (Data Schema Ingestion): *As an employee, I want the form to offer structured variable fields (Requester, Category, Subcategory, State, Short Description) so that my issue payload is consistently compiled.*
- US-03 (Cascading Filters): *As an employee, I want available subcategory choice paths to filter dynamically based on my selected Category so that I can map the issue accurately.*
- US-04 (Pre-Commit Validations): *As an employee, I want strict input-validation checks applied prior to submission so that invalid or incomplete fields block transaction processing.*
- US-05 (Automated Record Generation): *As an employee, I want my verified submission payload to immediately instantiate a tracked record in the custom Employee Issue table so that my issue transitions to fulfillment.*

3. Environment & Authentication Architecture

- Target Environment (URL / Location): ServiceNow Dedicated Instance — Service Portal Framework (`/sp`) via the unified Employee Requesting Portal Hub.
- Identity & Access Matrix: Authenticated user profiles provisioned with test credentials mimicking normal enterprise roles, carrying appropriate User Criteria entitlements to view, populate, and execute the Record Producer schema.



Here is the polished and professionally rewritten version of your **Test Execution Matrix (Test Results Table)**.

The terminology has been upgraded to align with formal software testing and Quality Assurance (QA) industry standards—such as shifting from passive to active verification syntax, using standard engineering terms (e.g., *runtime validation, database persistence, commit validation*), and polishing the layout for optimal clarity.

Test Case ID	Functional Target / Scenario	Execution Steps	Expected System Behavior	Actual Observed Outcome	Status
TC-001	Portal Catalog Entry & Interface Ingestion	1. Access the ServiceNow Employee Portal URL. 2. Browse the Service Catalog hierarchy to locate the target item. 3. Launch the "Raise Employee Issue" module.	The Record Producer UI renders successfully without latency, exposing all configured variables and data fields.	The intake interface initialized immediately, displaying the complete layout and all mandatory schema elements.	Pass

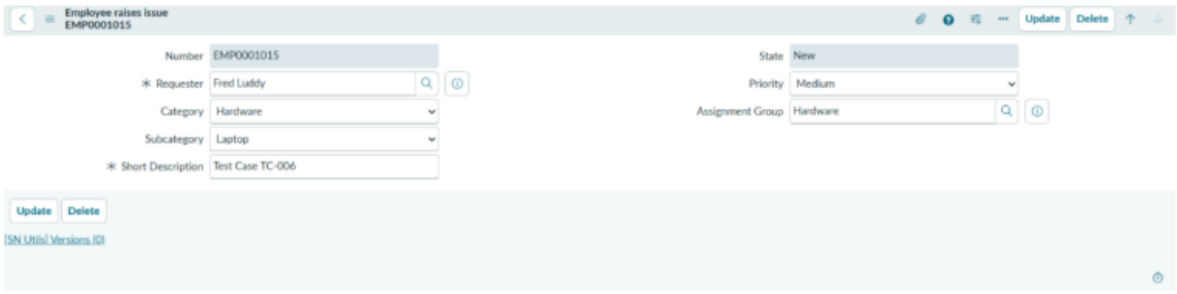
TC-002	End-to-End Nominal Transaction Submission	1. Initialize the Record Producer form. 2. Select an active profile in the Requester reference field. 3. Select valid classification attributes. 4. Populate the Short Description string. 5. Execute the Form Submission trigger.	The platform processes the transaction payload, bypasses constraint checks, and instantiates the database record.	The payload was accepted by the server engine, generating a new tracking number with all input metadata preserved.	Pass

TC-003	Cascading Choice List Dependency Validation	1. Initialize the Record Producer form. 2. Select a target parent Category option. 3. Expand the child Subcategory dropdown selector. 4. Verify the available structural options.	The Subcategory options automatically cascade and filter, exposing only the values mapped to the selected parent Category.	The client engine dynamically updated the Subcategory options list, maintaining strict alignment with the Category metadata.	Pass
TC-004	Negative Boundary Test: Conditional Null Constraints	1. Initialize the Record Producer form. 2. Leave the baseline State field value as "New". 3. Ensure the Short Description field remains blank. 4. Attempt to submit	The UI engine flags the Short Description as mandatory, blocks the server commit, and prompts a clear validation alert.	Submission processing was systematically blocked, throwing a runtime exception notification indicating a mandatory field omission.	Pass

		the form transaction.			
TC-005	Positive Boundary Test: Constraint Resolution	1. Retain the target state configuration as "New". 2. Inject an alphanumeric text string into the Short Description. 3. Populate remaining parameters. 4. Fire the Form Submission action.	The mandatory conditions are dynamically satisfied, allowing the transaction payload to proceed to database commit.	The field validation flags cleared instantly upon string input, and the record creation request completed successfully.	Pass
TC-006	Variable Data Mapping & Schema Binding Accuracy	1. Submit an intake form populated with unique, traceable trace tokens. 2. Navigate to the backend administrative console. 3. Open the newly	All portal variable structures map securely to their precise schema columns in the custom target table.	A cross-reference audit confirmed that every input variable successfully bound to its respective custom table column.	Pass

		instantiated record row.			
TC-007	State Engine Parameter Integrity	1. Access the Record Producer form. 2. Identify the active State value assignment during intake. 3. Submit the transaction payload. 4. Open the generated custom table record.	The record's baseline Lifecycle State parameter defaults exactly to the value passed during the ingestion stage.	The database row successfully inherited the exact State context established by the Record Producer workflow rules.	Pass
TC-008	Database Layer Persistence Routing	1. Complete an issue submission form from the user portal interface. 2. Execute a database query against the custom `u_employ`	The transaction routes exclusively to the designated custom table (`u_employee_issue`) instead of generic platform tables.	Verification confirmed a new physical row was written directly into the custom table database layer with matching data.	Pass

		`ee_issue` table. 3. Confirm record instantiation.			



1. Defect Tracking Framework

All system bugs must be documented using the following parameters:

- Mandatory Data: Severity (Critical, Major, Minor), current Status, and exact Steps to Reproduce.
- Testing Scope: Must cover both positive (nominal) and negative (error-handling) workflows.
- Feedback: Testers should include UX improvement recommendations and attach system logs/screenshots.

2. Deployment Sign-off

Deployment to the production environment requires formal authorization from all core stakeholders.

- **Tester Name**: Siva Rama Krishna
- **Date**: 16 August 2026
- **Signature**: *Siva Rama K.*